Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

Tutorial 8 : Premier projet Edge Impulse de bout en bout avec la Portenta H7 et OpenMV

Séries de tuto sur Portenta H7, lecture 8 « Edge Impulse, plateforme MLOps pour le TinyML »

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

2 novembre 2026

Objectifs du Tutorial

Dans ce tutoriel, vous allez construire en une séance le classifieur de matériau de la station EcoTri 4.0 avec Edge Impulse : collecte des images avec la Portenta, conception de l'impulse, apprentissage par transfert, test, déploiement OpenMV et mesures sur la carte. L'objectif est de parcourir tout le cycle une première fois, en reliant chaque écran du Studio aux notions du cours.

À la fin de ce tutoriel, vous serez capable de :

- Créer un projet Edge Impulse et y téléverser un jeu d'images capturé avec OpenMV IDE.
- Concevoir une impulse (entrée 96×96, bloc Image en niveaux de gris, transfer learning).
- Lire l'explorateur de caractéristiques, la matrice de confusion et les estimations de la cible.
- Déployer le modèle INT8 sous forme de bibliothèque OpenMV et l'exécuter sur la Portenta.
- Mesurer la latence réelle et la comparer à l'estimation du Studio.
- Créer une version du projet.

Durée indicative : 1 h 30. Ce tutoriel prépare le TP8 (détecteur FOMO multi-cibles, jalon J8).

1 Introduction sur Edge Impulse

Edge Impulse est une plateforme web de MLOps dédiée au TinyML (Hymel et al., MLSys 2023). Elle couvre l'acquisition et l'étiquetage des données, la conception d'une **impulse** (prétraitement + modèle), l'apprentissage, l'optimisation (EON Tuner, EON Compiler) et la génération de code pour de nombreuses cartes, dont la **Portenta H7** via **OpenMV**.

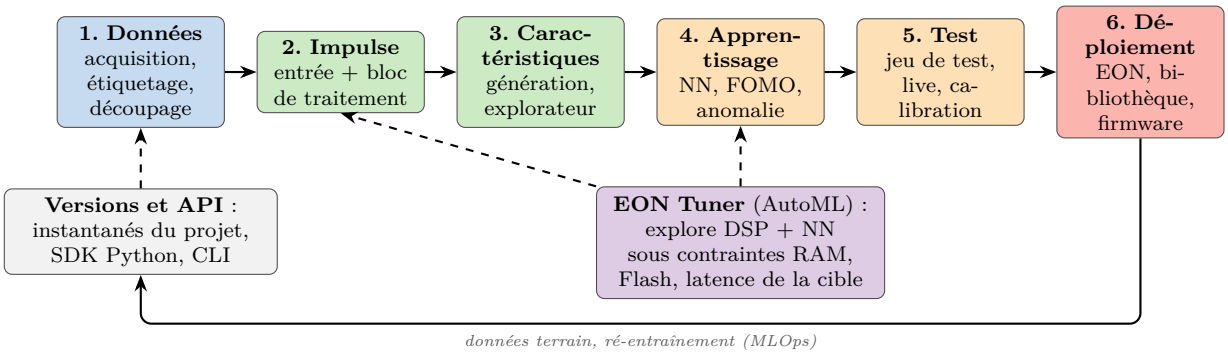
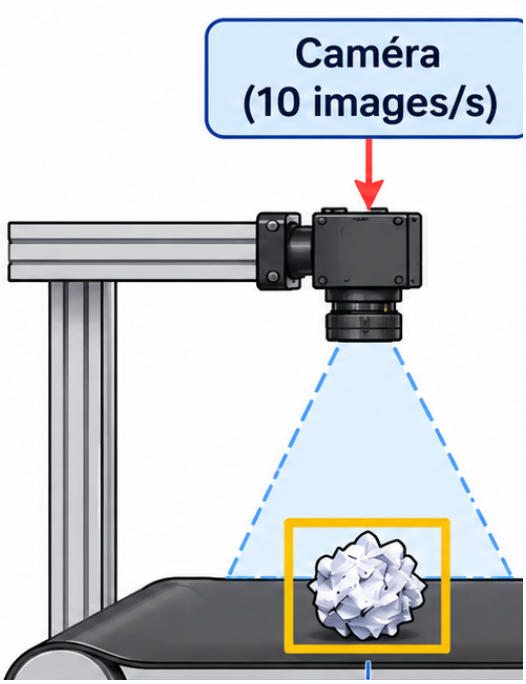


FIGURE 1 – *
Le flux de travail que nous allons suivre. Chaque bloc correspond à un onglet du Studio : vous les parcourrez dans l'ordre, de gauche à droite.

Ce que nous allons traiter



Dans la station EcoTri 4.0, la Portenta observe un objet à la fois sous la caméra et doit dire son matériau : plastique, verre, carton ou métal. Nous construisons ce classifieur avec Edge Impulse, puis nous l'exécutons sur la carte.

Rappel du cours : l'impulse

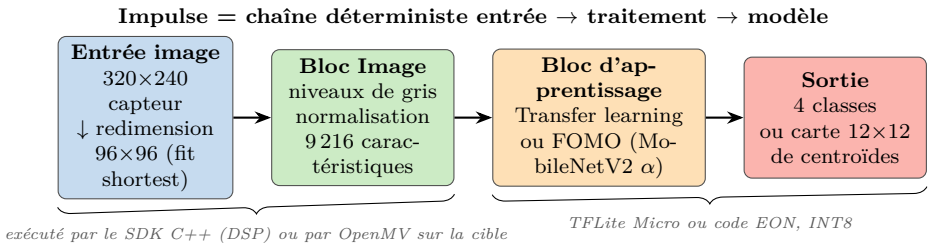


FIGURE 2 – *
L'impulse décrit toute la chaîne. Ce qui est fait à l'entraînement (taille, couleur, recadrage) doit être refait **à l'identique** sur la Portenta.

2 Pré-requis

Matériel

- Carte **Arduino Portenta H7** et **Vision Shield**, câble USB-C (données).
- Une vingtaine d'objets de déchets propres : bouteilles et barquettes plastiques, bocaux en verre, morceaux de carton, canettes.
- Un fond uniforme (feuille ou morceau de tapis) et un éclairage stable.

Logiciel

1. **OpenMV IDE** (version 4.x) avec le firmware OpenMV à jour sur la Portenta (Tutorial 1).
2. **Compte Edge Impulse** gratuit : studio.edgeimpulse.com.
3. Optionnel : **Edge Impulse CLI** (Node.js) : `npm install -g edge-impulse-cli`.

3 Étapes détaillées du tutoriel

Étape 1 : Créer le projet Edge Impulse

1. Connectez-vous au Studio et cliquez sur **Create new project**. Nommez-le `EcoTri-Materiau-<equipe>`.
2. Dans **Dashboard** → **Project info**, choisissez **Labeling method : One label per data item** (classification).
3. Dans le menu **Devices** (en haut à droite, sélecteur de cible), choisissez **Arduino Portenta H7 (Cortex-M7 480 MHz)** : les estimations de RAM, Flash et latence seront calculées pour cette carte.
4. Notez l'**identifiant du projet** (Project ID) et, dans **Keys**, l'existence de la clé d'API (ne la diffusez pas).

Étape 2 : Capturer le jeu de données avec OpenMV IDE

L'éditeur de jeu de données d'OpenMV enregistre les images **avec le capteur de déploiement**, ce qui évite un décalage de domaine.

1. Dans OpenMV IDE : **Tools** → **Dataset Editor** → **New Dataset**, choisissez un dossier vide `ecotri_dataset`.
2. Créez quatre classes avec le bouton **New Class Folder** : `plastique`, `verre`, `carton`, `metal`.

3. Remplacez le script `dataset_capture_script.py` proposé par celui-ci, qui fixe la taille et le recadrage :

```
import sensor, time

sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)     # capteur monochrome du Vision Shield
sensor.set_framesize(sensor.B320X320)
sensor.set_windowing((240, 240))           # image carrée centrée : pas de déformation
sensor.skip_frames(time=2000)
clock = time.clock()

while True:
    clock.tick()
    img = sensor.snapshot()
    print(clock.fps())
```

4. Lancez le script ; sélectionnez une classe dans le panneau de gauche, placez un objet sous la caméra puis cliquez sur **Capture Data** (icône appareil photo). Visez **40 à 60 images par classe**, en variant l'objet, sa position, son orientation et légèrement l'éclairage.

Astuce

Changez d'objet toutes les cinq ou six images. Dix photos presque identiques du même objet n'apportent presque rien et créent des fuites entre apprentissage et test.

Étape 3 : Téléverser les images dans le projet

Deux méthodes équivalentes :

1. **Depuis OpenMV IDE** : **Tools** → **Dataset Editor** → **Export** → **Upload to Edge Impulse Project**, connectez-vous et choisissez votre projet. Le nom de chaque dossier devient l'étiquette.
2. **En ligne de commande** (à partir du dossier du jeu de données) :

```
edge-impulse-uploader --category split --label plastique plastique.class/*.jpg
edge-impulse-uploader --category split --label verre     verre.class/*.jpg
edge-impulse-uploader --category split --label carton    carton.class/*.jpg
edge-impulse-uploader --category split --label metal     metal.class/*.jpg
```

L'option `-category split` répartit automatiquement 80 % en apprentissage et 20 % en test. Vérifiez dans **Data acquisition** la répartition par classe (*Train / Test split*) et supprimez les images floues ou vides.

Question

Vos images ont été prises en rafales d'un même objet. Pourquoi la répartition aléatoire image par image risque-t-elle de surestimer la précision ? Comment l'éviter ?

Étape 4 : Concevoir l'impulse

Ouvrez **Impulse design** → **Create impulse** :

1. **Image data** : largeur et hauteur **96**, *Resize mode* : **Fit shortest axis** (nos images sont déjà carrées).
2. **Add a processing block** : **Image**.
3. **Add a learning block** : **Transfer Learning (Images)**.

4. Cliquez sur **Save Impulse**.
5. Dans l'onglet **Image**, réglez *Color depth* sur **Grayscale**, puis **Save parameters** et **Generate features**.

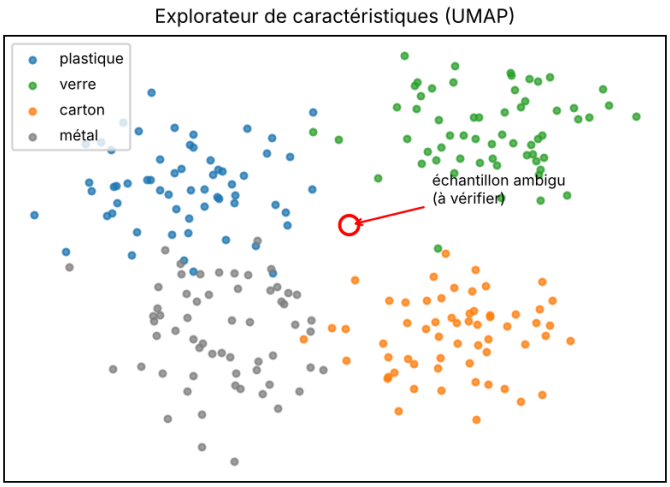


FIGURE 3 – *
Exemple d'explorateur de caractéristiques. Cliquez sur un point isolé pour afficher l'image correspondante : c'est souvent une erreur d'étiquette ou un objet ambigu.

Question

Combien de caractéristiques le bloc Image produit-il ? Relevez la RAM et le temps estimés pour ce bloc sur la Portenta (encadré *On-device performance*).

Étape 5 : Entraîner par transfert

Dans l'onglet **Transfer learning** :

1. Modèle : **MobileNetV2 96x96 0.35** (bouton *Choose a different model*).
2. Paramètres : 30 cycles d'apprentissage, taux d'apprentissage 0,0005, couche dense finale de 16 neurones, *dropout* 0,1, **Data augmentation** activée.
3. Lancez **Start training**. Une fois terminé, sélectionnez la version **Quantized (int8)** dans le panneau des résultats.

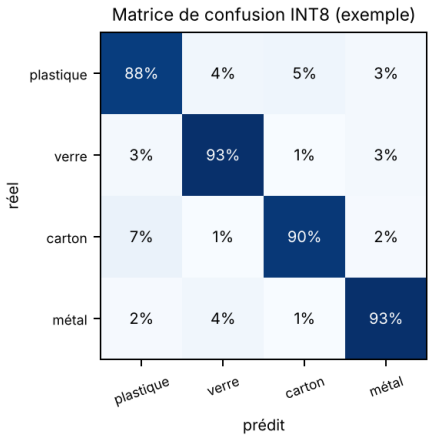


FIGURE 4 – *
Exemple de matrice de confusion. Repérez la paire de classes la plus confondue et cherchez la raison dans les images (forme, reflet, couleur perdue en niveaux de gris).

Modèle	**Précision validation**	**RAM estimée**	**Flash estimée**	**Latence estimée**
MobileNetV2 0.35, float32				
MobileNetV2 0.35, int8				
MobileNetV2 0.1, int8				

TABLE 1 – *
Tableau à compléter (refaites l'apprentissage avec $\alpha = 0{,}1$ pour la dernière ligne).

Étape 6 : Tester sur le jeu de test

Ouvrez **Model testing** puis **Classify all**. Comparez la précision sur le test à celle de la validation.

Question

Un écart de plus de 10 points entre validation et test : que soupçonnez-vous ? (Pensez au découpage par rafales et à l'éclairage des deux séries de capture.)

Étape 7 : Déployer la bibliothèque OpenMV

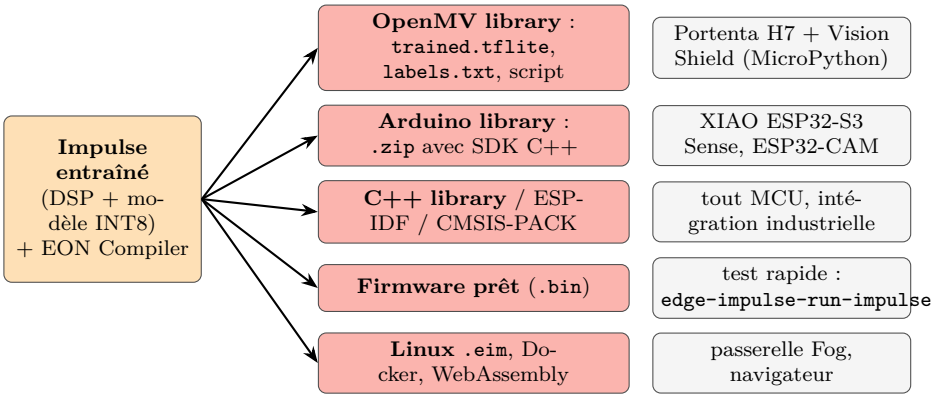


FIGURE 5 – *
Nous utilisons ici la première sortie : la bibliothèque OpenMV pour la Portenta.

1. Ouvrez **Deployment**, cherchez **OpenMV library**, laissez **Quantized (int8)** sélectionné (OpenMV n'accepte que les modèles INT8) et cliquez sur **Build**.
2. Décompressez l'archive : elle contient `trained.tflite`, `labels.txt` et `ei_image_classification.py`.
3. Copiez `trained.tflite` et `labels.txt` à la racine du disque USB de la Portenta, puis éjectez proprement le disque.

Étape 8 : Exécuter et mesurer sur la Portenta

Ouvrez le script suivant dans OpenMV IDE. Il reprend exactement le prétraitement de l'impulse, affiche le matériau et mesure séparément la capture et l'inférence.

```
import sensor, time, ml, gc

sensor.reset()
sensor.set_pixformat(sensor.GRAYSCALE)
sensor.set_framesize(sensor.B320X320)
sensor.set_windowing((240, 240))          # même recadrage que pour le dataset
sensor.skip_frames(time=2000)

net = ml.Model("trained.tflite", load_to_fb=True)
labels = [l.rstrip("\n") for l in open("labels.txt")]
print(net)                                # forme d'entrée et de sortie du modèle
gc.collect(); print("Tas libre :", gc.mem_free(), "o")

SEUIL = 0.70
while True:
    t0 = time.ticks_us()
    img = sensor.snapshot()
    t1 = time.ticks_us()
    scores = net.predict([img])[0].flatten().tolist()   # redimensionnement 96x96
    inclus
    t2 = time.ticks_us()
    k = scores.index(max(scores))
    decision = labels[k] if scores[k] >= SEUIL else "DOUTE"
    img.draw_string(4, 4, "%s %.2f" % (decision, scores[k]), color=255, scale=2)
    print("%-9s %.2f | capture %5.1f ms | inference %5.1f ms"
          % (decision, scores[k], time.ticks_diff(t1, t0) / 1000, time.ticks_diff(t2,
    t1) / 1000))
```

— `net.predict()` redimensionne l'image vers l'entrée du modèle (96×96) et applique la quantification d'entrée.

— Si la carte signale un manque de mémoire, retirez `load_to_fb=True` ou passez au modèle $\alpha = 0{,}1$.

— Le seuil produit la décision `DOUTE` vue à la lecture 6.

Grandeur	Estimation Studio	Mesure (IDE actif)	Mesure (IDE désactivé)
Temps d'inférence (ms)			
Temps de capture (ms)	—		
Tas libre après chargement (o)	—		

TABLE 2 – *
Estimation contre mesure : à reporter dans le TP8.

Question

L'estimation du Studio est calculée pour un code compilé, alors qu'OpenMV exécute le modèle avec TFLite Micro. Dans quel sens attendez-vous l'écart ? Vérifiez.

Étape 9 : Boucler : données de terrain et version

1. Avec la classification en direct, repérez trois objets mal classés ; capturez-les de nouveau avec le Dataset Editor et téléversez-les dans le projet.
2. Ré-entraînez et notez l'évolution de la précision sur le test.
3. Ouvrez **Versions** et cliquez sur **Create new version** avec la description « T8 : MobileNetV2 0.35 int8, 96x96 gris ». C'est le premier jalon de reproductibilité de votre projet.

Astuce

Le bouton **Export** du Dashboard permet de télécharger le jeu de données et les modèles : gardez une copie locale à chaque version.

4 Pour aller plus loin

— Edge Impulse, *Run on OpenMV* : docs.edgeimpulse.com.

— Arduino, *Training a Custom Machine Learning Model for Portenta H7* : docs.arduino.cc.

— Documentation du module `ml` d'OpenMV : docs.openmv.io.

— Harvard *Machine Learning Systems*, labs de M. Rovai (UNIFEI) : mlsysbook.ai.